S. Santhiya
192511860

SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO4:
Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards
(BL2,BL6)

Assessment Tool Weightage: 40%

Annexure A

79%

Coding challenge

1. Develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch. The program should accept the number of hosts as input and display the topology either in ASCII format or a graphical representation. For each host, print the host label, the name of the central switch, and the corresponding Cat6 UTP cable length connecting the host to the switch. Verify that all hosts are connected through the central switch and explain how the star topology facilitates reliable communication in a local area network.

2. Write a Python function named validate_segment(length_m) to verify whether a given **UTP cable segment** satisfies the IEEE 802.3 Ethernet standard, which specifies a maximum cable length of **100 meters**. The function should return **True** for valid cable lengths and return **False** along with an appropriate error message for invalid lengths exceeding the permissible limit. Demonstrate the correctness of the program by executing it with at least **three different test cases** and interpret the results.

3. Develop a Python program to simulate the operation of the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol in a star topology consisting of four hosts connected through a central switch. The program should simulate frame transmission attempts, detect transmission collisions, calculate random back-off intervals, and continue transmission after the waiting period. Record every collision event, transmission attempt, and back-off interval in a log file named csma_log.txt, and analyze how CSMA/CD minimizes data transmission conflicts in Ethernet networks.

4. Develop a client-server application using Python's socket library to simulate communication between two hosts connected through a central switch in a star topology. The server should receive frames from the client, forward them appropriately, and maintain a simulated MAC Address Table. After each successful frame transmission, display the updated MAC address table showing the learned MAC addresses and associated ports. Explain how MAC learning and frame forwarding are performed in Ethernet switches.

5. Design and implement a Python class named StarSwitch that models the basic functionality of an Ethernet switch. The class should include methods such as add_port(host_id) to connect a host, remove_port(host_id) to disconnect a host, and broadcast(frame) to transmit a frame to all connected ports except the source port. The program should maintain a record of active ports and log every broadcast operation, indicating the source host and the destination hosts that successfully receive the frame. Demonstrate the implementation using multiple hosts and explain the significance of broadcasting in Ethernet communication.

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem Understanding	15% 1.5	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
Algorithm Design	20% 2	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
Code Quality	15% 1.5	Clean, well-organized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
Functionality	20% 2	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases
Error Handling	10% 1	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10% 1	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
Documentation & Comments	5% 0.5	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments
Testing & Debugging	5% 0.5	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done

	A	B	C	D	E	F	G	H	
Q1	1	1.5	1	2	1	0.5	0.5	0.5	8
Q2	1	1.5	1	2	1	0.5	0.5	0.5	8
Q3	1.5	1	1.5	1	0.5	1	0.5	0.5	7.5
Q4	1.5	1	1.5	1	0.5	1	0.5	0.5	7.5
Q5	1	2	1	1.5	1	1	0.5	0.5	8.5

39.5

SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO4:
Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards
(BL2,BL6)

Assessment Tool Weightage: 40%

---

1. Develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch. The program should accept the number of hosts as input and display the topology either in ASCII format or a graphical representation. For each host, print the host label, the name of the central switch, and the corresponding Cat6 UTP cable length connecting the host to the switch. Verify that all hosts are connected through the central switch and explain how the star topology facilitates reliable communication in a local area network.

Python Program – Star Topology

Aim

To develop a Python program that generates and displays a Star Topology for a given number of hosts connected to a central switch, showing the host labels, switch name, and Cat6 UTP cable length, and to verify that all hosts are connected through the central switch**.**

Sample input

```
# Star Topology Generator
print ("===== STAR TOPOLOGY =====")

# Get number of hosts
n = int (input ("Enter the number of hosts: "))
```

```
switch = "Central Switch"

print("\topology:")
print ("            [Central Switch]")
print ("             /    |    \\")
print ("            /     |     \\")

# Display connections
for i in range (1, n + 1):
    cable length = 5 + (i * 2) # Example cable length in meters
    print(f'Host{i} -------- Cat6 UTP ({cable_length} m) --------> {switch}")




# Verification
print ("\n===== CONNECTION VERIFICATION =====")

connected = 0

for i in range(1, n + 1):
    cable_length = 5 + (i * 2)
    print(f'Host{i} -> {switch} : Connected "
        f"(Cat6 UTP Cable = {cable_length} m)")
    connected += 1

if connected == n:
    print ("\nVerification Successful!")
    print (f'All {n} hosts are connected through the central switch.")
else:
    print ("\nVerification Failed!")
```

Sample output:

```
===== STAR TOPOLOGY =====
Enter the number of hosts: 4
```

Topology:

[Central Switch]

/ | \

/ | \

Host1 -------- Cat6 UTP (7 m) --------> Central Switch

Host2 -------- Cat6 UTP (9 m) --------> Central Switch

Host3 -------- Cat6 UTP (11 m) -------> Central Switch

Host4 -------- Cat6 UTP (13 m) -------> Central Switch

===== CONNECTION VERIFICATION =====

Host1 -> Central Switch : Connected (Cat6 UTP Cable = 7 m)

Host2 -> Central Switch : Connected (Cat6 UTP Cable = 9 m)

Host3 -> Central Switch : Connected (Cat6 UTP Cable = 11 m)

Host4 -> Central Switch : Connected (Cat6 UTP Cable = 13 m)

Verification Successful!

All 4 hosts are connected through the central switch.

Explanation:

- The program asks the user to enter the number of hosts.
- A central switch is used to connect all the hosts.
- Each host is connected to the switch using a Cat6 UTP cable.
- The program displays the host name, switch name, and cable length.
- It checks whether all hosts are connected to the switch.
- If all hosts are connected, it displays “Verification Successful!”
- Star topology is reliable because if one cable or host fails, the other hosts can still communicate**.**

Result:

The Python program was successfully developed and executed. It accepts the number of hosts as input, displays the Star Topology, shows the Cat6 UTP cable length for each host, and verifies that all hosts are connected through the central switch.

2.Write a Python function named validate_segment(length_m) to verify whether a given UTP cable segment satisfies the IEEE 802.3 Ethernet standard, which specifies a maximum cable length of 100 meters**.** The function should return **True** for valid cable lengths and return False along with an appropriate error message for invalid lengths exceeding the permissible limit. Demonstrate the correctness of the program by executing it with at least three different test cases and interpret the results.

Answers:

Python Program – UTP Cable Segment Validation

Aim

To develop a Python function, validate segment(length_m) to verify whether a UTP cable segment satisfies the maximum 100-meter cable length specified for Ethernet, and to test the function using different cable lengths.

Sample input:

```
def validate_segment(length_m):
    # Check whether cable length is within the limit
    if length_m <= 100:
        return True
    else:
        return False, "Error: Cable length exceeds the maximum limit of 100 meters."

# Test cases
test_cases = [50, 100, 120]

for length in test_cases:
    result = validate_segment(length)
    print ("Cable Length:", length, "meters")
    if result is True:
        print ("Result: Valid")
        print("Message: Cable length is within the 100-meter limit.")
    else:
        print("Result: Invalid")
        print ("Message:", result [1])
```

```
print ()
```

Sample Output:

Cable Length: 50 meters

Result: Valid

Message: Cable length is within the 100-meter limit.

Cable Length: 100 meters

Result: Valid

Message: Cable length is within the 100-meter limit.

Cable Length: 120 meters

Result: Invalid

Message: Error: Cable length exceeds the maximum limit of 100 meters.

Test Cases and Interpretation

Test Case	Cable Length	Result	Interpretation
1	50 m	Valid	Less than 100 m, so it is acceptable.
2	100 m	Valid	Equal to the maximum limit, so it is acceptable.
3	120 m	Invalid	Exceeds 100 m, so an error message is displayed.

Result

The Python function was successfully developed and tested with three different cable lengths. 50 m and 100 m were identified as valid segments, while 120 m was rejected because it exceeds the 100-meter maximum limit.

3.Develop a Python program to simulate the operation of the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol in a star topology consisting of four hosts connected through a central switch. The program should simulate frame transmission attempts, detect transmission collisions, calculate random back-off intervals, and continue transmission after the waiting period. Record every collision event, transmission attempt, and back-off interval in a log file named csma_log.txt, and analyze how CSMA/CD minimizes data transmission conflicts in Ethernet networks.

Answers:

Python Program – CSMA/CD Simulation in Star Topology

Aim

To develop a Python program to simulate the CSMA/CD protocol for four hosts connected through a central switch, detect collisions, calculate random back-off intervals, and record transmission attempts, collisions, and back-off intervals in a log file named csma_log.txt.

Sample Input:

```
import random
import time
# Four hosts connected to a central switch
hosts = ["Host 1", "Host 2", "Host 3", "Host 4"]
switch = "Central Switch"
# Open log file
log_file = open ("csma_log.txt", "w")
log_file.write("========== CSMA/CD SIMULATION LOG ==========\n")
log_file.write(f"Topology: 4 Hosts connected to {switch}\n\n")
print ("========== CSMA/CD SIMULATION ==========")
print ("Topology: 4 Hosts -> Central Switch\n")
# Each host has one frame to transmit
pending_hosts = hosts.copy()
# Collision attempt count for each host
attempt_count = {host: 0 for host in hosts}
while pending_hosts:
    print("\nHosts ready to transmit:", pending_hosts)
    # Randomly decide which hosts attempt transmission
    transmitting = []
```

```
for host in pending_hosts:
    if random.choice([True, False]):
        transmitting.append(host)
# Make sure at least one host transmits
if not transmitting:
    transmitting.append(random. Choice(pending_hosts))
# Record transmission attempts
for host in transmitting:
    attempt_count[host] += 1
    message = (
        f"Transmission Attempt: {host} -> {switch}, "
        f"Attempt {attempt_count[host]}\n"
    )
    print(message.strip())
    log_file.write(message)
# Check for collision
if len(transmitting) > 1:
    collision_message = (
        f"COLLISION DETECTED between: "
        f"{', '.join(transmitting)}\n"
    )
    print(collision_message.strip())
    log_file.write(collision_message)
    # Apply random back-off
    for host in transmitting:
        k = attempt_count[host]
        # Binary exponential back-off
        max_slots = min (2 ** k - 1, 10)
        backoff = random.randint(0, max_slots)
        backoff_message = (
            f"{host}: Back-off interval = {backoff} slot(s)\n"
        )
        print(backoff_message.strip())
        log_file.write(backoff_message)
    print ("Hosts wait for the back-off period and retry.\n")
    log_file.write(
        "Hosts wait for the back-off period and retry.\n\n"
```

```
        )

        time.sleep(1)
    else:
        # Successful transmission
        successful_host = transmitting[0]
        success_message = (
            f"SUCCESS: {successful_host} transmitted the frame "
            f"without collision.\n"
        )
        print (success_message. strip())
        log_file.write(success message)
        # Remove successfully transmitted host
        pending_hosts. remove (successful host)
        time.sleep(1)
# Close log file
log_file.write("\n========== SIMULATION COMPLETED ==========\n")
log_file.close()
print("\n========== SIMULATION COMPLETED ==========")
print ("All four hosts successfully transmitted their frames.")
print ("Transmission details are stored in csma_log.txt")
```

Sample output:

```
========== CSMA/CD SIMULATION ==========
Topology: 4 Hosts -> Central Switch

Hosts ready to transmit: ['Host 1', 'Host 2', 'Host 3', 'Host 4']

Transmission Attempt: Host 1 -> Central Switch, Attempt 1
Transmission Attempt: Host 3 -> Central Switch, Attempt 1
COLLISION DETECTED between: Host 1, Host 3

Host 1: Back-off interval = 1 slot(s)
Host 3: Back-off interval = 0 slot(s)

Hosts wait for the back-off period and retry.
```

Transmission Attempt: Host 3 -> Central Switch, Attempt 2
SUCCESS: Host 3 transmitted the frame without collision.

Transmission Attempt: Host 1 -> Central Switch, Attempt 2
SUCCESS: Host 1 transmitted the frame without collision.

Transmission Attempt: Host 2 -> Central Switch, Attempt 1
SUCCESS: Host 2 transmitted the frame without collision.

Transmission Attempt: Host 4 -> Central Switch, Attempt 1
SUCCESS: Host 4 transmitted the frame without collision.

========== SIMULATION COMPLETED ==========
All four hosts successfully transmitted their frames.
Transmission details are stored in csma_log.txt

Explanation:

- **Hosts:** Four hosts are created and connected to the central switch.
- **Transmission:** Each host tries to send data.
- **Collision:** If two or more hosts send data at the same time, a collision occurs.
- **Back-off:** After a collision, the hosts wait for a random time.
- **Retry:** After waiting, the hosts try to send the data again.
- **Success:** If only one host sends, the data is transmitted successfully.
- **Log File:** All attempts, collisions, and back-off times are stored in csma_log.txt.
- **Purpose:** Random waiting helps reduce repeated collisions and improves network communication.

Result:

The Python program was successfully developed and executed to simulate **CSMA/CD** for four hosts. The program detected collisions, generated random back-off intervals, retransmitted the frames, and recorded all events in **`csma_log.txt`**. Thus, the program successfully demonstrated how CSMA/CD reduces data transmission conflicts.

4.Develop a client-server application using Python's socket library to simulate communication between two hosts connected through a central switch in a star topology. The server should receive frames from the client, forward them appropriately, and maintain a simulated MAC Address Table. After each successful frame transmission, display the updated MAC address table showing the learned MAC addresses and associated ports. Explain how MAC learning and frame forwarding are performed in Ethernet switches.

Answers:

Python Client-Server Application – MAC Address Learning

Aim

To develop a client-server application using Python socket programming to simulate communication between two hosts connected through a central switch, maintain a simulated MAC address table, and demonstrate MAC learning and frame forwarding.

Sample Input:

```
import socket
# Central switch MAC address table
mac_table = {}
# Create server socket
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
HOST = "127.0.0.1"
PORT = 5000
server.bind((HOST, PORT))
server.listen(1)
print ("==================================")
print ("   STAR TOPOLOGY - CENTRAL SWITCH")
print ("==================================")
# Host details
hosts = {
   "Host 1": {
      "mac": "AA:BB:CC:DD:EE:01",
```

```
        "port": "Port 1"
    },
    "Host 2": {
        "mac": "AA:BB:CC:DD:EE:02",
        "port": "Port 2"
    }
}
# Display topology
print("\topology:")
print ("            Central Switch")
print ("             /          \\")
print ("          Host 1       Host 2")
print ("         Port 1       Port 2")
# Simulate frame transmission
for host_name, details in hosts.items():
    print("\n-----------------------------------")
    print("Frame received from:", host_name)
    source_mac = details["mac"]
    source_port = details["port"]
    # MAC learning
    mac_table[source_mac] = source_port
    print ("Source MAC:", source_mac)
    print("Incoming Port :", source_port)
    # Display MAC table
    print ("\nUpdated MAC Address Table")
    print ("-----------------------------------")
    print ("MAC Address\t\tPort")
    for mac, port in mac_table.items():
        print (mac, "\t", port)
    # Frame forwarding
```

```
    if host_name == "Host 1":
        destination = "Host 2"
        destination_mac = hosts["Host 2"]["mac"]
        destination_port = hosts["Host 2"]["port"]
    else:
        destination = "Host 1"
        destination_mac = hosts["Host 1"]["mac"]
        destination_port = hosts["Host 1"]["port"]
    print ("\nFrame Forwarding")
    print ("Destination :", destination)
    print ("Destination MAC :", destination_mac)
    print ("Forwarded through:", destination_port)
print("\n===================================")
print ("MAC learning and frame forwarding completed.")
print ("===================================")
server.close()
```

Sample Output:

```
  STAR TOPOLOGY - CENTRAL SWITCH
Topology:
            Central Switch
             /          \
          Host 1        Host 2
         Port 1        Port 2
Frame received from: Host 1
Source MAC: AA:BB:CC:DD:EE:01
Incoming Port: Port 1
Updated MAC Address Table
MAC Address              Port
AA:BB:CC:DD:EE:01        Port 1
Frame Forwarding
```

Destination : Host 2

Destination MAC : AA:BB:CC:DD:EE:02

Forwarded through : Port 2

Frame received from: Host 2

Source MAC : AA:BB:CC:DD:EE:02

Incoming Port: Port 2

Updated MAC Address Table

MAC Address Port

AA:BB:CC:DD:EE:01 Port 1

AA:BB:CC:DD:EE:02 Port 2

Frame Forwarding

Destination: Host 1

Destination MAC : AA:BB:CC:DD:EE:01

Forwarded through: Port 1

MAC learning and frame forwarding completed.

Explanation

- Host 1 and Host 2 are connected to the Central Switch.
- Each host has a unique MAC address.
- When the switch receives a frame, it learns the source MAC address and its port.
- The MAC address and port are stored in the MAC Address Table.
- The switch checks the destination MAC address and forwards the frame through the correct port.
- Thus, the program demonstrates MAC learning and frame forwarding in an Ethernet switch.

Result

The Python program was successfully developed to simulate a star topology with two hosts and a central switch. The program learned the MAC addresses, updated the MAC address table, and forwarded frames through the appropriate ports.

5.Design and implement a Python class named StarSwitch that models the basic functionality of an Ethernet switch. The class should include methods such as add_port(host_id) to connect a host, remove_port(host_id) to disconnect a host, and broadcast(frame) to transmit a frame to all connected ports except the source port. The program should maintain a record of active ports and log every broadcast operation, indicating the source host and the destination hosts that successfully receive the frame. Demonstrate the implementation using multiple hosts and explain the significance of broadcasting in Ethernet communication.

Answers:

Python Program – StarSwitch

Aim

To design and implement a Python class named StarSwitch that simulates the basic functions of an Ethernet switch, including connecting hosts, disconnecting hosts, broadcasting frames, and maintaining active ports.

Sample Input:

```
class StarSwitch:
    def __init__(self):
        # Store active hosts and their ports
        self.active_ports = {}
        # Store broadcast logs
        self.broadcast_log = []
    # Method to connect a host
    def add_port(self, host_id):
        if host_id not in self.active_ports:
            port_number = len(self.active_ports) + 1
            self.active_ports[host_id] = f"Port {port_number}"
            print(f"{host_id} connected to {self.active_ports[host_id]}")
        else:
            print(f"{host_id} is already connected.")
    # Method to disconnect a host
    def remove_port(self, host_id):
```

```
        if host_id in self.active_ports:
            port = self.active_ports.pop(host_id)
            print(f"{host_id} disconnected from {port}")
        else:
            print(f"{host_id} is not connected.")
    # Method to broadcast a frame
    def broadcast(self, frame):
        source = frame["source"]
        data = frame["data"]
        print ("\n========== BROADCAST ==========")
        print ("Source Host :", source)
        print ("Frame       :", data)
        destinations = []
        # Send frame to all hosts except source
        for host in self.active_ports:
            if host != source:
                destinations.append(host)
                print (f'Frame sent to {host}")
        # Save broadcast information
        log = {
            "source": source,
            "destinations": destinations,
            "frame": data
        }
        self.broadcast_log.append(log)
        print ("Destination Hosts:", destinations)
# Create a StarSwitch object
switch = StarSwitch()
# Add multiple hosts
switch.add_port ("Host 1")
```

```
switch.add_port ("Host 2")
switch.add_port ("Host 3")
switch.add_port ("Host 4")
# Display active ports
print ("\n========== ACTIVE PORTS ==========")
for host, port in switch.active_ports.items():
    print (host, "->", port)
# Broadcast a frame from Host 1
frame1 = {
    "source": "Host 1",
    "data": "Hello from Host 1"
}
switch.broadcast(frame1)
# Broadcast another frame from Host 3
frame2 = {
    "source": "Host 3",
    "data": "Hello from Host 3"
}
switch. Broadcast(frame2)
# Remove a host
print ("\n========== REMOVE HOST ==========")
switch.remove_port ("Host 4")
# Display updated active ports
print ("\n========== UPDATED ACTIVE PORTS ==========")
for host, port in switch.active_ports.items():
    print (host, "->", port)
# Display broadcast log
print ("\n========== BROADCAST LOG ==========")
for log in switch.broadcast_log:
    print ("Source:", log["source"])
```

```
    print ("Destinations:", log["destinations"])
    print ("Frame:", log["frame"])
    print ()
```

Sample Output:

```
Host 1 connected to Port 1
Host 2 connected to Port 2
Host 3 connected to Port 3
Host 4 connected to Port 4
========== ACTIVE PORTS ==========
Host 1 -> Port 1
Host 2 -> Port 2
Host 3 -> Port 3
Host 4 -> Port 4
========== BROADCAST ==========
Source Host : Host 1
Frame       : Hello from Host 1
Frame sent to Host 2
Frame sent to Host 3
Frame sent to Host 4
Destination Hosts: ['Host 2', 'Host 3', 'Host 4']
========== BROADCAST ==========
Source Host : Host 3
Frame       : Hello from Host 3
Frame sent to Host 1
Frame sent to Host 2
Frame sent to Host 4
Destination Hosts: ['Host 1', 'Host 2', 'Host 4']
========== REMOVE HOST ==========
Host 4 disconnected from Port 4
========== UPDATED ACTIVE PORTS ==========
```

Host 1 -> Port 1

Host 2 -> Port 2

Host 3 -> Port 3

========== BROADCAST LOG ==========

Source: Host 1

Destinations: ['Host 2', 'Host 3', 'Host 4']

Frame: Hello from Host 1

Source: Host 3

Destinations: ['Host 1', 'Host 2', 'Host 4']

Frame: Hello from Host 3

Explanation:

- StarSwitch represents the central Ethernet switch.
- add_port () connects a host to the switch.
- remove_port () disconnects a host.
- broadcast() sends a frame to all active hosts except the source host.
- active_ports stores the currently connected hosts.
- broadcast_log records every broadcast operation.

Result:

The StarSwitch class was successfully implemented. Multiple hosts were connected and disconnected, frames were broadcast to all active hosts except the source, and each broadcast operation was recorded in the log.

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
Code Quality	15%	Clean, well-organized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases
Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments
Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done